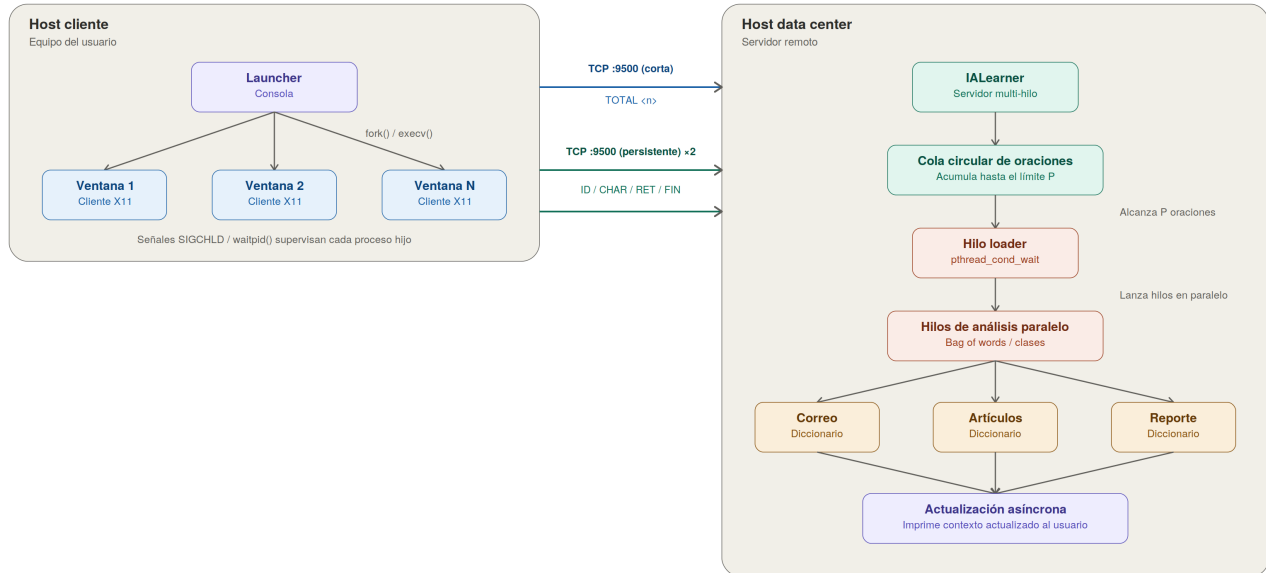


Documento de Diseño — Agentic-OS

Autor: LUIS ERNESTO ROCA MACIAS

Diagrama de Despliegue



2. Arquitectura General

El sistema Agentic-OS emplea una arquitectura distribuida separada en dos entornos físicos o lógicos: el Host Cliente (equipo del usuario) y el Host Data Center (servidor remoto). En el Host Cliente, el proceso Launcher interactúa con el usuario y orquesta la creación de N procesos hijos (Ventana X11). En el Data Center, el proceso IA Learner actúa como un servidor TCP multi-hilo que recibe la información en tiempo real de cada ventana de forma asíncrona. Esta separación garantiza que el procesamiento pesado de clasificación textual no afecte el rendimiento de la interfaz gráfica del usuario.

3. Mecanismos de Multiprogramación y Concurrency (Eficiencia)

El diseño garantiza un alto rendimiento mediante el uso de multiprocesamiento local y concurrencia avanzada en el servidor:

Local (Launcher y Ventanas): La generación de ventanas se realiza mediante llamadas a `fork() + execv()`, permitiendo que cada interfaz gráfica corra en su propio espacio de memoria. Para la gestión del ciclo de vida, el Launcher implementa un manejador de señales `SIGCHLD` acoplado a un bucle `waitpid(-1, &estado, WNOHANG)`. Esto permite recolectar múltiples procesos hijos que terminan simultáneamente sin bloquear la ejecución del menú interactivo y previniendo la aparición de procesos zombie.

Remoto (Procesamiento por Lotes y Hilo Loader): En el servidor IA Learner, las oraciones completadas no se procesan de forma inmediata y aislada, sino que se encolan en una estructura compartida protegida por exclusión mutua (`pthread_mutex_t`).

Variables de Condición y Hilo Loader: Un hilo especializado denominado Loader monitorea la cola compartida mediante `pthread_cond_wait`. Al acumularse el límite exacto del parámetro P de oraciones (independientemente de la ventana de origen), el hilo Loader se despierta y crea de manera dinámica un conjunto de P hilos en paralelo para procesar el lote completo de forma simultánea.

4. Mecanismos de IPC (Comunicación entre Procesos)

Dado que los procesos residen en hosts distintos, la comunicación por tuberías locales es inviable; por ello, se implementó un mecanismo IPC basado en sockets TCP. El Launcher establece una conexión corta por el puerto 9500 exclusivamente para transmitir el comando `TOTAL <n>`, informando al servidor la cantidad inicial de ventanas. Paralelamente, cada proceso Ventana X11 abre una conexión persistente hacia el IALearner por el mismo puerto. Se diseñó un protocolo de texto ligero basado en prefijos de una línea terminados en `\n` (ID, CHAR, RET, FIN), lo que minimiza la sobrecarga de red y permite una transmisión carácter por carácter altamente eficiente.

5. Algoritmos de Clasificación y Evaluación Asíncrona

El procesamiento del texto entrante utiliza la técnica Bag of Words. Las oraciones formadas tras presionar Enter se transforman en vectores de frecuencias que se evalúan contra tres diccionarios en memoria (Correo, Artículo, Reporte).

Regla de pertenencia: Un documento se clasifica dentro de una categoría si presenta coincidencias con al menos tres palabras distintas del diccionario correspondiente. En caso de calificar para múltiples clases, el algoritmo de desempate selecciona aquella con la mayor suma total de frecuencias.

Inferencia Asíncrona de Usuario: Conforme los hilos del lote P analizan las oraciones en paralelo, el sistema evalúa de forma asíncrona las reglas de la tabla de decisión (Administrativo, Técnico, Profesor, Estudiante), actualizando el perfil del usuario en tiempo real en la consola del servidor.

6. Programación Defensiva y Liberación de Recursos

La robustez del código se asegura mediante prácticas defensivas en todos los componentes:

Ambos procesos principales validan estrictamente sus parámetros de entrada (como el valor numérico de P) y no sufren caídas abruptas ante configuraciones inválidas.

El cierre de recursos es explícito: todos los descriptores de sockets y displays de X11 se cierran correctamente, y los mutex y variables de condición se destruyen al finalizar la ejecución del servidor.

El sistema de apagado del Launcher envía la señal `SIGTERM` para un cierre coordinado, escalando a `SIGKILL` como medida de respaldo para procesos bloqueados, liberando completamente la memoria y la CPU.